Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore
Université Côte d'Azur, Inria, France

Enrico Tassi
Université Côte d'Azur, Inria, France

Abstract

We mechanize two operational semantics for PROLOG with cut and prove them equivalent, in the Rocq prover. The first semantics closely models the ELPI's implementation, it is based on a stack of choice points and is well suited for studying optimizations employed by PROLOG abstract machines. The second semantics is instead based on and-or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics (1) to prove properties about the effect of the cut operator in the evaluation of choice points, since the rules from classical logic do not apply, and (2) to reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the "UCA DS4H" project, reference ANR-17-EURE-0004.

1 Introduction

ELPI is a dialect of λPROLOG (see [18, 19, 8, 13]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [14, 3, 4, 25, 9, 10]. Other remarkable libraries include the Hierarchy-Builder library-structuring tool [5] and Derive [23, 24, 12], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI is equipped with a static analysis for determinacy [11] to help users control backtracking. ROCQ users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows the user to assert which predicates should be considered deterministic and then checks that these predicates behave like functions, i.e., they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point correspond to an suspended *alternative* in the execution trace. In the following we use *choice point* and *alternative* as synonyms.

This paper reports our first steps towards a mechanization, in the ROCQ prover, of the determinacy analysis from [11]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI's implementation and is similar to the semantics mechanized by Pusch in ISABELLE/HOL [20] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations

```
func f int -> int. % f is a deterministic predicate. The two rules
f 1 2.             % have non-verlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int. % r is a non-deterministic predicate. The heads
r 0 0.             % overlap: the query for 0 has three applicable
r 0 1.             % rules.
r 0 2 :- !.
func g int -> int. % g is deterministic: if the first rule succeeds, the
g A C :- r A B, f B C, !.  % second one is cut away, as well as the choice
g D F :- f D E, f E F.     % points left by the call to r.
```

Figure 1 Running example of a ELPI program

used by standard PROLOG abstract machines [26, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive querys. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1.

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [11]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. Firstly, at most one rule should be used on a given query. This condition is satisfied if (i) the rules have non-overlapping heads: if two heads do not overlap in the inputs they accept, then no query can unify with both, or if (ii) we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded. Secondly, each rule of a deterministic predicate should (i) either call functions, this ensure that no choice point is left after the execution of the premises of the chosen rule, (ii) or call relations provided that they are followed by the cut operator, so that any allocated choice points preceeding the cut are discarded.

We show that if every rule defining a deterministic predicate passes this analysis, then any call to that predicate leaves no choice points.

2 Preliminaries: syntax and informal semantics of cut

In the entire paper we use Figure 1 as our running example, and we start by describing how we represent a program like that one. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ-calculus style (no parentheses).

The description of a program is given by the record $\mathbb{P}$ and is shared by both semantics. A program consists of a list of rules `R` together with a signature environment `sigT`. We delay the descussion of signatures to Section 6 where we describe the static analysis that concerns them.

```
Inductive Tm := Symb of S | Pred of P | Var of V | App of Tm & Tm.
Inductive Atom := cut | call of Tm.
Record R := mkR { head : Tm; premises : list Atom }.
Record ℙ := { rules : seq R; sig : sigT }.
Definition Σ := {fmap V -> Tm}.
```

Figure 2 Definition of term, rule, program and substitution

A rule consists of a head term `Tm` and a list of premises. Each premise is an `Atom`, i.e. either the cut operator or a predicate call. Terms `Tm` include predicate symbols, data symbols, and variables. The syntax tree allows variables to be applied and predicates to be mixed with data. These higher order features play no role in the current paper that focuses on first order logic programming, but allows future extensions to full λProlog to be based on the same mechanization.

The set of variables `V`, predicates `P` and data symbols `S` are countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the finmap library [17]. The empty substitution is written ε while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$. usiamo + ?

The backchaining operation applies all rules to a query term; it is central to both semantics.

```
Definition bc : ℙ → 𝓕ᵥ → Tm → Σ → 𝓕ᵥ * list (Σ * list Atom)
```

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_\nu$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization, we abstract over the unifier and its properties. The function `unify` takes two terms t_1 and t_2, together with a substitution σ, as input, and returns an optional substitution σ' that extends σ. The expected behavior of `unify` is higher-order unification restricted to the pattern fragment, as explained in [18]. Therefore, if t_1 and t_2 unify under σ, then $\sigma'\ t_1 \equiv \sigma'\ t_2$, where $\sigma\ t$ is the notation for subsitution application and $\equiv$ denotes the syntactic equality of its arguments.

2.1 The cut operator

There are quite a few variants of the cut operator. The one we are interested in, i.e. the one used in Elpi, is known as *hard cut* (see, e.g., the documentation of SWI-Prolog [22]). Whenever the `backchain` function returns a non-empty list of alternatives, the first one is explored immediately, while the others are suspended as *choice points*. Within a given alternative, premises are processed from left to right, executing each atom in turn. When a cut is encountered, it discards (i) all choice points created by `backchain` for the current call and (ii) all choice points created while executing the premises to the left of the cut.

For example, consider the program in Figure 1 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\mathtt{r\ A\ B,\ f\ B\ C,\ !}]),\ (\sigma_2, [\mathtt{f\ D\ E,\ f\ E\ F}])]$$

with $\sigma_1 := \{\mathtt{A} \mapsto 0, \mathtt{C} \mapsto \mathtt{R}\}$ and $\sigma_2 := \{\mathtt{D} \mapsto 0, \mathtt{F} \mapsto \mathtt{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_\nu$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1, i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [!])]$ where $\sigma_3 := \sigma_1 + \{\mathtt{B} \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{\mathtt{B} \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{\mathtt{B} \mapsto 2\}$.

The next step of interpretation continues with `f B C` under σ_3, that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [!])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4, i.e. `f 1 R`. This succeeds with $\sigma_6 := \sigma_4 + \{\mathtt{R} \mapsto 2\}$.

```
Inductive tree ≔
  | KO | OK                                  (* failure and success *)
  | Todo of Atom                            (* unexplored branch *)
  | Or   of option tree & Σ & tree          (* A ∨ B_σ *)
  | And  of tree & list Atom & tree.        (* A ∧_{B_0} B *)
```

Figure 3 Definition of And-Or tree

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for `r` and one from the second rule for `g` (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for `g` was selected. (In contrast, a cut does not discard choice points created earlier; in this example, there are none.)

The final solution is: $\{$`A` $\mapsto$ `0`, `B` $\mapsto$ `1`, `C` $\mapsto$ `R`, `R` $\mapsto$ `2`$\}$.

In Sections 3 and 4, we provide fully mechanized operational semantics for Prolog with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

Notational conventions In the following section we note $\mathbb{B}$ for the bolean type, and $\top$ and $\bot$ for `true` and `false`. The constructors of the option type are written $\cdot t$ for `Some` t and $\square$ for `None`. Following the Mathematical Components library, on which we depend, we use `x.1` and `x.2` to denote the first and second projection applied to the pair `x`.

spiegare notation for list

3 And-Or Tree semantics

TODO: nested option, output Fv also inside option corner cases: prog2.elpi

An And-Or tree is a stratified, variable-width tree. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

The tree is built incrementally. The `Todo` node represents an unexplored branch (an `Atom`). When the atom is a `call`, we use the backchaining procedure from Section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the `step` procedure described in Section 3.2.

The Rocq data type for And-Or trees is given in Section 3. In addition to `Todo`, it provides two distinguished nodes, `KO` and `OK`, which represent respectively the smallest failed and successful tree.

The `Or` node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A. The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ, which becomes the current substitution when backtracking to B.

why option and not KO?

The `And` node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first choice point in A. We call B_0 the *reset point* for B: it represents the continuation for all alternatives of A except the first. That is, when backtracking occurs within A, the subtree B is replaced by the atoms in B_0. An example is shown in Section 3.3.

A graphical representation of a tree is shown in Figure 6b. For compactness, we depict `And` and `Or` nodes as n-ary (rather than binary), with children associating to the right. The

```
Inductive tag ≔ Expanded | CutBrothers | Failed | Success.
Definition step : ℙ → 𝓕𝓿 → Σ → tree → (𝓕𝓿 * tag * tree)
Definition prune : 𝔹 → tree → option tree
Inductive runT : ℙ →  𝓕𝓿 → Σ → tree → option (Σ * option tree) → 𝔹 → 𝓕𝓿 → Prop
```

Figure 4 Signatures of the tree-based semantics

KO node acts as the terminating element of an Or list, while OK is the terminating element of an And list.

3.1 The tree-based semantics

The tree is constructed incrementally by two recursive functions, `step` and `prune`. Both traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved thanks to the `next_tree` (in Figure 5). When this node is OK we say the entire tree is a *success*, when it is KO we say the tree *failed*, otherwise the tree is *incomplete*.

Since all functions must terminate in Rocq, we define their transitive closure of `step` and `prune` as the inductive relation `runT`. More precisely `runT` iterates calls to `step` on incomplete trees to expand `Todo` nodes. When a tree fails it calls `prune` to find the next alternative and continues from there, if one exists. We detail `step`, `prune`, and `runT` in Sections 3.2–3.4.

In Figure 6 we mark with a black arrow the result of `next_tree`.

3.2 The `step` procedure

The `step` procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a `tag`, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

The `tag` (see Section 3.1) indicates which kind of step was performed. `Failure` and `Success` are returned for trees that satisfy, respectively, the `failed` and `success` tests. `CutBrothers` indicates the interpretation of a superficial cut: `step` was called on an And-layer and its

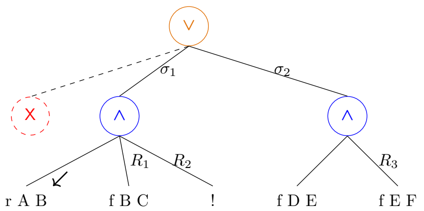
[1] The substitutions $\sigma_1 \dots \sigma_6$ are from Section 2.1. R_1, R_2, and R_3 are reset points and correspond respectively to the lists of atoms [`f Y Z, !`], [`!`], and [`f Y Z`].

```
Fixpoint next σ t : Σ * tree :=
  match t with
   | Todo _ | KO | OK ⇒ (σ, t)
   | Or □ σ′ B ⇒ next σ′ B
   | Or ·A _ _ ⇒ next σ A
   | And A _ B ⇒
     let (σ′, pA) := next σ A in
     if pA == OK then next σ′ B
     else (σ′, pA)
  end.
```

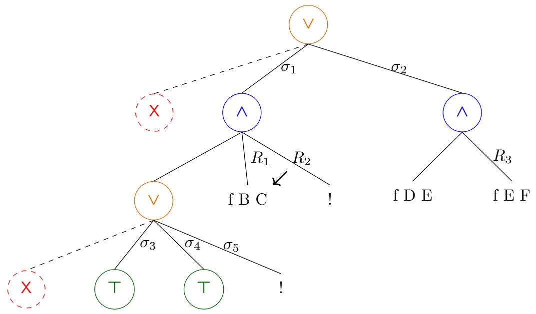
```
Definition next_subst σ t := (next σ t).1.
Definition next_tree t := (next ε t).2.

Definition success t := next_tree t == OK.
Definition failed t := next_tree t == KO.
Definition incomplete t :=
    if next_tree t is Todo _ then ⊤ else ⊥.
```

Figure 5 `next` and derived functions g 0 R

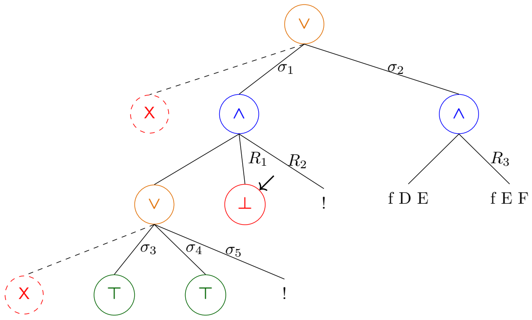
(a) Initial query



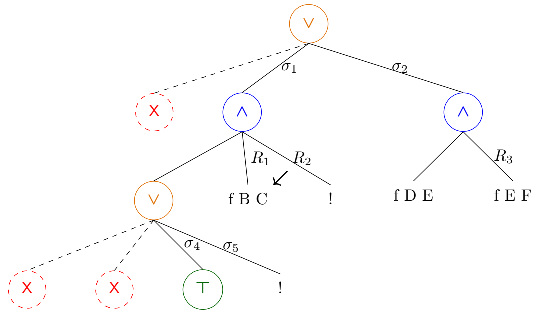
(b) Tree after `step` on `g 0 R`



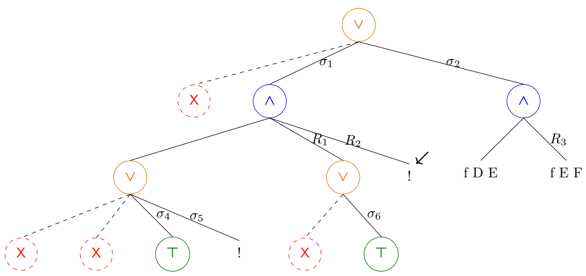
(c) `step` on Figure 6b



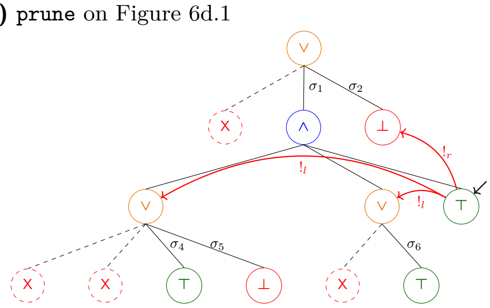
(d.1) `step` on Figure 6c



(d.2) `prune` on Figure 6d.1



(e) `step` on Figure 6d.2



(f) `step` on Figure 6e

Figure 6 Some tree representations[1] `next_tree` is the cut atom. Finally, `Expanded` accounts for the interpretation of predicate calls and non-superficial cuts.

The `step` procedure evaluates atoms. In particular, it leaves the tree unchanged when the tree is *success* or *failed*.

▶ **Lemma 1** (succ_step_iff). ∀ *p v σ t, success t* ↔ *step p v σ t = (v, Success, t).*

▶ **Lemma 2** (fail_step_iff). ∀ *p v σ t, failed t* ↔ *step p v σ t = (v, Failed, t).*

The `step` procedure expands `Todo` nodes by calling `backchain`, which returns a list l of alternatives. If l is empty, then the current call atom is replaced by `KO`. Otherwise, it is replaced by a right-skewed tree of `Or` nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is `OK`; otherwise, it is a right-skewed tree of `And` nodes.

Figure 6 depicts the tree corresponding to the running example (Section 2.1) as it undergoes calls to `step`. We run query `q 0 R` against the program P in Figure 1. Let c_0, sottolineare le
c_1, and c_2 be the children of the root. By construction, c_0 is the ☐ tree, while c_1 and c_2 sostituzioni
correspond respectively to the premises of rules r_1 and r_2 in P. Note that c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in Section 2.1. Reset points are defines as follows: R_1 := [`f Y Z, !`], R_2 := [`!`], and R_3 := `f Y Z`. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of `step` performing a tree expansion via backchain appear in Figures 6c–6e.

3.2.1 The interpretation of a cut

The step corresponding to a cut performs two actions: (i) the cut node is replaced by `OK`; and (ii) the subtrees in the scope of `Cut` are pruned. More precisely, (ii) prunes the left siblings of the `Cut` node (in the same And-layer, denoted $!_l$) and prunes the right uncles of `Cut` (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in Figure 6f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. We note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by `KO`. This amounts to discarding the third rule of predicate `r`.

3.3 The `prune` procedure

When `backchain` returns an empty list, `step` produces a *failed* tree and the computation must backtrack. The `prune` procedure is responsible for producing the new tree from which the computation can continue.

In Figure 6d.1 we encounter a failure because no rule applies to the atom `f B C` under substitution σ_3, which maps `B` to `0`. The `prune` procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the `OK` node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the `KO` node with the information stored in the reset point R_1. This restores the call `f B C`, which can then be reconsidered under substitution σ_4.

Furthermore, `prune` serves another important purpose. Its behavior depends on the boolean flag it receives as input: `prune` $\perp$ t recursively removes all failures (if any) from t, whereas `prune` $\top$ t removes the first success in t. For example, the tree in Figure 6f contains a successful path that maps `R` to `2`. Calling `prune` on this tree returns ☐, since there are no alternatives in the tree after the success.

$$\frac{\texttt{success}\ t \qquad \texttt{next\_subst}\ \sigma\ t = \sigma' \qquad \texttt{prune}\ \top\ t = t'}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ \cdot(\sigma', t')\ \bot\ v}\ \mathit{StopT}$$

$$\frac{\begin{array}{cc}\texttt{incomplete}\ t & b' = (\mathrm{tg} = \texttt{CutBrothers}) \vee b \\ \texttt{step}\ \mathrm{p}\ v\ \sigma\ t = (v', \mathrm{tg}, t') & \texttt{runT}\ \mathrm{p}\ v'\ \sigma\ t'\ r\ b\ v''\end{array}}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ r\ b'\ v''}\ \mathit{StepT}$$

$$\frac{\texttt{failed}\ t \qquad \texttt{prune}\ \bot\ t = \cdot t' \qquad \texttt{runT}\ \mathrm{p}\ v\ \sigma\ t'\ r\ \mathrm{n}\ v'}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ r\ \mathrm{n}\ v'}\ \mathit{BackT}$$

$$\frac{\texttt{prune}\ \bot\ t = \square}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ \square\ \bot\ v}\ \mathit{FailT}$$

Figure 7 Rule system for `runT`

Some interesting properties of `prune` are shown below; they illustrate how `prune` complements `step` with respect to failed, successful, and incomplete trees.

▶ **Lemma 3** (incomplete_prune_id). $\forall\ b\ t,\ \textit{incomplete}\ t \to \textit{prune}\ b\ t = \cdot t.$

▶ **Lemma 4** (prune_Some). $\forall\ b\ t\ t',\ \textit{prune}\ b\ t = \cdot t' \to \textit{failed}\ t' = \bot.$

▶ **Lemma 5** (prune_None). $\forall\ t,\ \textit{prune}\ \bot\ t = \square \to \textit{failed}\ t.$

3.4 The `runT` inductive

The inductive relation `runT` relates four inputs to three outputs. The inputs are a program, a set of variables, an initial substitution σ, and a tree t. The outputs are (i) an optional result r, (ii) a boolean b, and (iii) an updated set of variable names.

The main result r is $\square$ when the execution fails; otherwise when $r = \cdot(\sigma', t')$ the execution succeeded and produced the solution σ'; t' represents the remaining, not-yet-explored alternatives. The boolean b records whether a superficial cut was evaluated during execution.

The `runT` predicate has four cases, depicted as inference rules in Figure 7. (*StopT*) If `next_tree` t is a success, then the substitution σ' is extracted from t (starting from σ) and the input tree is cleaned of its successful path. (*StepT*) If `next_tree` t is an atom, then `step` is invoked to evaluate t, and `runT` is called recursively on the updated tree. (*BackT*) If `next_tree` t is a failure, then `prune` is called to clear the failed path; if the resulting cleaned tree exists, `runT` is called recursively on it. Finally, (*FailT*) accounts for trees with no solutions.

The set of rules defining `runT` is deterministic.

▶ **Lemma 6** (runT_det). $\forall\ p\ v_0\ \sigma\ t\ r\ r'\ b\ b'\ v\ v',\ \textit{runT}\ p\ v_0\ \sigma\ t\ r\ b\ v \to \textit{runT}\ p\ v_0\ \sigma\ t\ r'\ b'\ v' \to r' = r \wedge v' = v \wedge b' = b.$

The proof is by induction on the first `runT` derivation and is immediate, because the rules are selected based on `next_tree` t. In particular, the hypotheses `success` t, `incomplete` t, and `failed` t are mutually exclusive. Moreover, by **??**, the hypothesis of *FailT* implies *failed* t; hence *FailT* is exclusive with *StopT* and *StepT*, and it is also exclusive with *BackT* since they impose different requirements on the output of `prune`.

The boolean output of `runT` allows state interesting properties about the `Or` node in presence of cut. Here we only report a selection of the ones we proved:

▶ **Lemma 7** (runSST_or). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ l\ l'\ \sigma_1\ r,\ \textit{runT}\ p\ v\ \sigma\ l\ \cdot(\sigma',\ \cdot l')\ \top\ v' \to$

$\texttt{runT}\ p\ v\ \sigma\ (\cdot l\ \vee\ r_{\sigma_1})\ \cdot(\sigma',\ \cdot(\cdot l'\ \vee\ \texttt{KO}_{\sigma_1}))\ \perp\ v'.$

▶ **Lemma 8** (runNT_or). $\forall\ p\ v\ v'\ \sigma\ t\ \sigma_1\ t',\ \texttt{runT}\ p\ v\ \sigma\ t\ \square\ \top\ v' \rightarrow$
$\texttt{runT}\ p\ v\ \sigma\ (\cdot t\ \vee\ t'_{\sigma_1})\ \square\ \perp\ v'.$

▶ **Lemma 9** (run_orSST). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ l\ l'\ r\ r',\ \texttt{runT}\ p\ v\ \sigma\ (\cdot l\ \vee\ r_{\sigma_1})\ \cdot(\sigma',\ \cdot(\cdot l'\ \vee\ r'_{\sigma_1}))\ \perp\ v' \rightarrow$
$\exists\ b,\ \texttt{runT}\ p\ v\ \sigma\ l\ \cdot(\sigma',\ \cdot l')\ b\ v'\ \wedge\ r' = \texttt{if}\ b\ \texttt{then}\ \texttt{KO}\ \texttt{else}\ r.$

Lemmas 7 and 8 correspond to the introduction rules for disjunction when the first disjunct performs a cut. If A executes a cut one *cannot* obtain $A \vee B$ out of B since the execution of A, be it in the end succesful or not, makes the success of B irrelevant (the cut discards it). In the first lemma B is forced to be `KO`, while in the latter the failure of A absorbs B. Depicted as rules:

$$\frac{A \qquad (A \text{ cuts})}{A \vee \top} \qquad \frac{\neg A \qquad (A \text{ cuts})}{\neg(\neg A \vee B)} \qquad \frac{B \qquad (A \text{ does not cut})}{A \vee B}$$

Finally, Lemma 9 is an elimination rule for disjunction. From $A \vee B$, where A is not inhert (i.e. already explored) we cannot deduce A or B independently. We deduce only $\top \vee B'$ where B' provides no information if A performed a cut.

$$\frac{A \vee B \qquad \begin{matrix}[\mathrm{A}]\\ C\end{matrix} \qquad (A \text{ cuts})}{C} \qquad \frac{A \vee B \qquad \begin{matrix}[\mathrm{A}]\\ C\end{matrix} \qquad \begin{matrix}[B]\\ C\end{matrix} \qquad (A \text{ does not cut})}{\mathrm{C}}$$

As anticipated in the intructions, these lemmas disprove the commutativity of disjunction.

4 Stack semantics

We now introduce the stack semantics. The interpreter is close to that of the ELPI language and provides an operational semantics that is similar to the one proposed by Pusch in [20], which is in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [26, 1].

We represent a state of the ELPI language thanks to the two mutual inductives shown below.

```
Inductive alts  := nilA | consA of (Σ * goals) & alts
with      goals := nilG | consG of (Atom * alts)  & goals .
```

A stack is an enhanced two-dimensional list, called *alts* in the inductive. The outermost list represents a list of alternatives in disjunction, each accompanied by the substitution that should be used for their interpretation. The innermost list is a list of atoms, that is, the list of goals in conjunction. These goals are decorated with a pointer to an alternative list, which is used to keep track of where to jump when a cut is interpreted. We call these special alternatives the *cut-to* alternatives.

The stack interpreter is called **runS** and has the following type.

```
Inductive runS (p: ℙ) : 𝓕_𝓥 → alts → option (Σ * alts) → Prop        (1)
```

The inductive **runS** represents a routine taking as input a set of variables and a list of alternatives and returning an option. $\square$ stands for interpretation with no solution, whereas $\cdot(\sigma, a)$ represent a successful interpretation with σ as solution and a as the list of non-yet-explored choice points.

The rule system for **runS** is given in Figure 8 and is made by 4 cases. We distinguish two main cases: if the list of alternatives is empty, (*FailS*) we have a failure: we stop the computation and return $\square$. If the list of alternatives is the list $(\sigma_0, h) :: a$, the interpreter evaluates the first choice point h under the substitution σ_0. If h is empty, (*StopS*), then we have a success, i.e. we return the input σ_0 and leave a as the future choice points. Otherwise, h is the list $(t, ca) :: g$ where t is the first atom to evaluate, ca is its cut-alternative list and g is the list of future conjuncts. If t is a cut (*CutS*), we need to keep evaluating g under σ_0 but we change the alternatives to ca. Finally, in

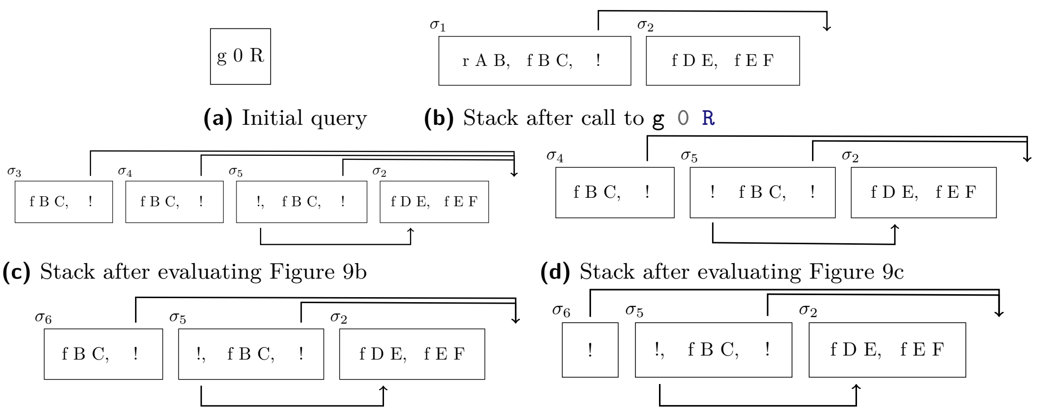
```
Definition save_gs a g b :=
  [seq (x, a) | x <- b] ++ g.

Definition save_as a g b :=
  [seq (x.1, save_gs a g x.2) | x <- b].
```

```
Definition stepS p v t σ a g ≔
  let (v′, r) ≔ backchain p v t σ in
  (v′, save_as a g r).
```

$$\frac{}{\texttt{runS}\ \mathrm{p}\ v\ ((\sigma,\ \varnothing)\ ::\ a)\ \cdot(\sigma,\ a)}\ \textit{StopS}
\qquad
\frac{\texttt{runS}\ \mathrm{p}\ v\ ((\sigma,\ g)\ ::\ ca)\ \mathrm{r}}{\texttt{runS}\ \mathrm{p}\ v\ ((\sigma,\ (\mathrm{cut},\ ca)\ ::\ g)\ ::\ _)\ \mathrm{r}}\ \textit{CutS}$$

$$\frac{\texttt{stepS}\ \mathrm{p}\ v\ t\ \sigma\ a\ g = (v_1,\ a') \qquad \texttt{runS}\ \mathrm{p}\ v_1\ (a' \mathbin{+\!\!+} a)\ \mathrm{r}}{\texttt{runS}\ \mathrm{p}\ v\ ((\sigma,\ (\mathrm{call}\ t,\ _)\ ::\ g)\ ::\ a)\ \mathrm{r}}\ \textit{CallS}
\qquad
\frac{}{\texttt{runS}\ \mathrm{p}\ _\ \varnothing\ \square}\ \textit{FailS}$$

Figure 8 Rule system for `runS`



(a) Initial query **(b)** Stack after call to `g 0 R`
(c) Stack after evaluating Figure 9b **(d)** Stack after evaluating Figure 9c
(e) Stack after evaluating Figure 9d **(f)** Stack after evaluating Figure 9e

Figure 9 Example of `runS` execution the case of a call (*CallS*), we backchain the call in the program. If rs is the list of rule premises returned by `backchain`, then each list of premises if mapped to a list of goals (`goals`) have a as cut-alternative and the global list is translated to a list of alternative (`alts`).

Dire una parola su backtracking = tl if bc = nil

Example of execution We propose an example of the execution of `runS` in Figure 9. The arrows in the images represent the cut-alternative pointers of the cut. We have not drawn arrows coming out of atom-calls since the interpreter ignores the cut-alternatives in this case (cf. *CallS*). The evolution of the stack in Figure 9 mirrors the example in Section 2.1: we evaluate the first goal of the first alternative. During backchaining, we add a pointer to the remaining alternatives to each new cut operator. If a cut is evaluated, we replace the remaining alternatives with the cut-to alternatives. For example, in Figure 9f, evaluating the cut discards the second and third elements in the stack. Backtracking is performed by simply removing the first alternative from the stack, as shown in Figure 9d.

Remark We want finally to point out how the lemma that we were able to prove in Section 3.4 are not easy to be expressed in the stack semantics. This is beacuse there is no sufficient structure in the stack to understand what is the scope of the cut operator. For example, in a stack made of these alternatives $a_0, a_1 \ldots a_n$, we have no clear idea of what happens if the first alternative a_0 succeeds: there could be a cut in a_0 but, is this cut superficial? What is its impact wrt the alternatives $a_1 \ldots a_n$? More then that, it is also possible that the state is hill-formed and the cut does not point to a suffix in the list of alternatives.

5 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. In particular we define a translation from trees to stacks, called `t2l`. We do not explcitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [2].

Before describing the translation of trees to stacks (in Section 5.2) we need to define the notion of *valid tree*. The `tree` datatype indeed contains trees that cannot be generated by `runT` via `step` or `prune`, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

5.1 Valid trees (`valid_tree`)

The class of valid trees is characterized by the function shown in Figure 10a.

While all base cases of tree are considered valid, we need to analyze carefully the cases for the `Or` and `And` constructors.

For the `Or` constructor, we distinguish two cases depending on whether the left hand side is present. If it is not present, then the right hand side must be a valid tree. Otherwise, the the left hand side must be a valid tree and the right hand side must either be the `KO` tree or be unexplored. The first case occurs when the right hand side is cut out by the evaluation of a superficial cut present in the left hand side. In the latter case we use the `base_or` predicate to check that `B` is the right-skewed tree formed by a disjunction of conjunctions that is genereated after a succeful backchain to a `call`.

For the `And` constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. `B` must be the right-skewed tree containing conjunctions of premises atoms. This is enforced using the `big_and` produce that generates a tree out of the reset point B_0, the same procude used to transform each alternative output by `backchain` into the And-layer of the resulting tree. When the left hand side is successfule, then the right hand side may have been explored, and consequently must be a valid tree.

5.2 From trees to stacks (`t2l`)

The `t2l` recursive function translates a tree to a stack. For space constraints Figure 10b only gives the base cases, while we describe the recursive cases in the following text. DAVVERO?

```
Fixpoint valid_tree A :=
  match A with
  | Todo _ | OK | KO ⇒ ⊤
  | Or □ _ B ⇒ valid_tree B
  | Or ·A _ B ⇒ valid_tree A &&
       ((B == KO) || B.base_or B)
  | And A B0 B ⇒ valid_tree A &&
      if success A then valid_tree B
      else B == big_and B0
  end.
```

(a) Valid tree

```
Fixpoint t2l t σ (cp: alts) : alts :=
  match t with
  | OK     ⇒ [(σ, ∅)]
  | KO     ⇒ ∅
  | TA a   ⇒ [(σ, [(a, ∅)])]
  ...
```

(b) Tree to list

Figure 10 Valid tree and Tree to list

The function `t2l` takes as input the tree t_0, a substitution σ_0, and a list of choice points *cp*. These choice points represent alternatives that appear at the same level of the current tree, such as, for example, the right siblings of an `Or` node. The substitution is used to determine under which substitution the subtree being compiled lives.

The `OK` node represent one succeful alternative, thereofre it is translated into a singleton list with the empty list of goals. The `KO` node represent a failure, i.e. it is mapped to the empty list of alternatives. Finally, atoms are mapped to one alternative containing a single goal with the empty cut-to alternative list. They cannot point to *cp* since they are not right-uncle subtrees, i.e. *cp* are alternatives that will be actually discarded if the atom is a cut.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B. The alternatives of B are valid choice points for A and should be passed to the translation of A. The two list of alternatives can be concatenated and, each atom, should add *cp* as new cut-to alternatives: A and B are one level lower wrt *cp* and therefore the cut they have should point to *cp*.

superficial?

The translation of $A \wedge_{B_0} B$ starts with the translation of A. If the translation of A is an empty list we return the empty list of alternatives without even touching B nor B_0, since an empty translation for A indicates failure and this in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$ the B node represent the continuation of the first alternative g, whereas B_0 is the continuation of each alternative in a.

5.2.1 Example of `t2l`

The translation of the trees in Figure 6 is given in Figure 9. The letters in the figures relate the trees to the corresponding letters in the stack figures. For example, the tree 6b is translated into 9b. We note that the two trees 6d.1 and 6d.2 are both translated into 9d, showing that *t2l* is not injective; that is, there are different trees that map to the same stack. This means that there are transitions in `runT` that are not visible in `runS`.

As a more precise example of the behavior of `t2l`, we consider the tree 6c, and in particular its deepest `Or` node. This subtree shows a disjunction with four children. The first is ignored, since it is □. The success node below σ_3 has `f B C` and the cut operator as continuation, corresponding to the first cell in the stack in 9c. We note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals `f B C, !`; this is translated into the second cell of the stack in 9c. Finally, the cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative in Figure 9c. We can see that the first cut points to the translation of the subtree under σ_2, since it is one `Or`-layer above its right uncles.

5.3 Equivalence proof

The tree-based semantics exposes more information so to be usable to express the lemmas in Section 3.4. For the equivalence this information is irrelevant, hence define a version of `runT` that hides it.

▶ **Definition 10** (runT' (p v σ t r)). `∃ b v', runT p v σ t r b v'.`

From Figures 6 and 9, we observe that, upon backtracking, `runT` may perform more reduction steps than `runS` before returning a solution or a failure. These transitions are called silent transitions and must be skipped when proving the equivalence between the two semantics. Therefore, we state the following lemma.

▶ **Lemma 11** (pruneF_t2l).
```
∀ t t' b, valid_tree t → failed t → prune b t = ·t' →
 ∀ l σ, t2l t σ l = t2l t' σ l.
```

Proof. By induction on the tree t. ◀ dire di v

The first direction of the equivalence states that the execution of a valid tree is matched by the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives returned as a tree correspond to those returned as a stack.

▶ **Theorem 12** (tree_to_elpi).
```
∀ p t σ r, let v := vars_tree t `|` vars_sigma σ in
 valid_tree t →
   runT' p v σ t r →
     let a := t2l t σ ∅  in
     let r' :=  if r is ·(σ', t) then  ·(σ', t2l (odflt KO t) σ ∅ )
                else ☐ in
     runS p v a r'.
```

Proof. We proceed by induction on the execution of `runT'`. There are four cases to consider. The first case arises from *StopT*; it deals with successful trees. A successful tree must start with an empty list, and therefore we conclude using *StopS*. The case for *StepT* requires treating two subcases: `step` evaluates either a cut or a call. Accordingly, we apply *CutS* or *CallS*, followed by the induction hypothesis. The case for *BackT* is silent: it represents the non-injective behavior of `t2l`; however, thanks to Lemma 11 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation *FailT*. It is easily proved using *FailS* and the fact that if `prune` returns ☐ when trying to erase failure in t, then `t2l` applied to t returns the empty list. ◀

The converse direction is stated following [2]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns a unexplored tree that translates to the unexplored stack.

▶ **Theorem 13** (elpi_to_tree).
```
∀ p v a r, runS p v a r →
 ∀ σ t, valid_tree t → t2l t σ ∅  = a →
   if r is ·(σ', a') then
     ∃ t', runT' p v σ t ·(σ', t') ∧ t2l (odflt KO t') σ ∅  = a'
   else runT' p v σ t ☐.
```

Proof. The proof is based on the idea explained in [2, Section 3.5.1] and depicted in the figure on the right: we have a state t whose translation is a, we proceed by induction on the execution of `runS`. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to the stack state is a'. This proof strategy is indirectly providing a kind of `l2t` function by combining the execution of the `runS` and `t2l`. ◀

$$\begin{tikzcd} t \arrow[r, "\texttt{runT'}\ t"] \arrow[d, "\text{t2l}"'] & t' \arrow[d, "\text{t2l}"] \\ a \arrow[r, "\texttt{runS}\ a"'] \arrow[ur] & a' \end{tikzcd}$$

Stating the converse direction in a symmetric way would have requires a function `l2t` transforming an stack a to a tree t. Writing this function is far from trivial, since the structure of the tree is lost by `save_alts` and run that intuitively keep the state into a big disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover one would need to define a notion of valid stack that is equally intricate.

6 Case study: determinacy alanysis

An interesting application of the tree semantics is determinacy checking, introduced in version 3 of the ELPI language [11]. The checker takes as input the signatures of the predicates declared by the user and verifies that the rules of a deterministic predicate return at most one solution; we call this particular kind of predicate a function. Thanks to this static verification, the user can better control unexpected backtracking at compile time: if an inconsistency is detected, a fatal error explains why the current program may violate determinacy.

A function behaves deterministically if two main conditions are satisfied: *mutual exclusion* guarantees that at most one rule can be successfully applied to a given query, whereas *rule checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates.

Determinacy checking is strongly related to the modes of predicates: we differentiate between input and output arguments, where outputs are generated wrt the the received inputs. A deterministic call relates to its inputs only one output. In the literature, we find several works about determinacy checking, namely [21, 16, 7]. These works needs a mode checker statically ensuring that in a query with ground inputs, each recursive call is performed with ground inputs.

In our mechanization, and in particular in the ELPI language, we adopt a special unification routine on input argument, called `matching` [1, 27]. Similarly to `unify`, `matching` takes the query-term t_1 and the pattern t_2 and returns an optional subsitution σ'. The query-term is the argument in input mode taken from a dynamic call and the pattern is the corresponding input argument in the chosen rule-head.

The function `matching` has the following property:

▶ **Axiom 1** (matching_disj). $\forall\ t_1\ t_2\ \sigma\ \sigma'$, *[disjoint vars_tm* t_1 *& domf* σ*]* $\to$ *disjoint_tm* $t_1\ t_2 \to$ *matching* $t_1\ t_2\ \sigma$ = $\cdot\sigma' \to \exists$ *e, domf* σ' = *domf* σ *`|` e* $\wedge$ *e* $\leq$ *vars_tm* t_2.

If the two terms are disjoint and the variables in the support of σ are disjoint from the variables in t_1, then, if matching succeeds, only the variables in t_2 are assigned; that is, $t_1 \equiv \sigma'\ t_2$, and the variables in t_1 are not assigned in σ'.

The `bc` function uses `matching` or `unify` on the arguments q depending if the argument is in *input* or *output* mode. Below we axiomatize two important properties of our unificators.

▶ **Axiom 2** (unif_trans). $\forall\ t_1\ t_2\ t_3\ \sigma$, *unify* $t_1\ t_2\ \sigma \to$ *unify* $t_2\ t_3\ \sigma \to$ *unify* $t_1\ t_3\ \sigma$.

▶ **Axiom 3** (match_unif). $\forall\ t_1\ t_2\ \sigma$, *matching* $t_1\ t_2\ \sigma \to$ *unify* $t_1\ t_2\ \sigma$.

In Figure 1, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that `f` and `g` are expected to be functions, as specified by the `func` keyword, whereas `r` is a relation, as indicated by the `pred` keyword. The `->` symbol separates inputs from outputs; therefore, all three predicates are expected to take an `int` as input and return an `int` as output.

The signatures of the program are stored in the `sig` projection of the program P passed to the interpreter and are encoded as described in [11]; that is, similarly to the code in Figure 1, we use arrows for term application, data types to encode base types such as *int*, and two special symbols to distinguish deterministic from non-deterministic predicates.

A program execution behaves deterministically if, given a program, a substitution, and a tree to the `runT` inductive, the execution either fails or, if it succeeds, the tree of alternatives is $\square$. Below, we define `is_det`.

▶ **Definition 14** (is_det (p σ v t)). $\forall\ r,\ \mathit{runT'}\ p\ v\ \sigma\ t\ r \to r = \square \lor \exists\ \sigma,\ r = \cdot(\sigma,\ \square)$.

The theorem we want to prove is the following.

▶ **Theorem 15** (det_check_call). $\forall\ p\ \sigma\ t\ v,$

$\mathit{check\_}\mathbb{P}\ p \to \mathit{tm\_is\_det}\ p.(\mathit{sig})\ t \to \mathit{is\_det}\ p\ \sigma\ v\ (\mathit{Todo}\ (\mathit{call}\ t))$.

Here, `check_`$\mathbb{P}$ denotes the function that checks the program with respect to mutual exclusion and rule checking, and `tm_is_det` denotes the function that tests whether the head of the term refers to a rigid constant with a deterministic type.

The proof of this lemma should proceed by induction on the derivation of the program execution. However, with the current definition, the induction hypothesis is too weak: it would be formulated over a `Todo` tree, whereas the conclusion may concern an `Or` tree, rendering the induction hypothesis unusable.

To address this issue, we introduce the notion of "deterministic trees" (`det_tree`), show that a tree satisfying this condition enjoys the desired properties, and then prove the following theorem, which is more general than `det_check_call`.

▶ **Lemma 16** (det_check_tree). $\forall\ \sigma\ v\ p\ t,\ \mathit{check\_}\mathbb{P}\ p \to \mathit{det\_tree}\ p.(\mathit{sig})\ t \to \mathit{is\_det}\ p\ \sigma\ v\ t$.

Since `det_check_call` is an instance of `det_check_tree`, its proof is now trivial.

As an important remark, proving the same lemma with the stack semantics, it is difficult to correctly define the predicate `det_stack`, checking for the deterministc behavior of a generic stack, since, similar to Section 3.4, the lack of structure make this goal hard. We need therefore an intermediate data structure, like the tree to encompass this problem.

7 Related work

We studied a Prolog semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the Elpi interpreter, which is in turn inspired by B-Prolog's "matching-clauses" [1, 27]. The two semantics we presented (and proved equivalent) also apply to standard Prolog if one forces all predicate signatures to have only output arguments.

For determinacy analysis, matching input arguments affects the development only as follows: Axiom 1 does not require the query q to be ground. Adapting our results to standard Prolog therefore requires the insertion of two additional hypotheses in Theorem 15. First, we require that t has ground inputs. Second, we require that the program p is statically well-moded.

Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. Therefore, the proofs in Section 6 still apply.

The only mechanization of Prolog's semantics we could find is the one by Pusch in Isabelle/HOL [20] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their work start from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [26, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in Section 4 that pushed us to develop an more explicit semantics (with respect to cut). In some way our work is complementary, showing the stack based semantics equivalent to a more optmized one.

Another relevant work are the ones by King [15], but we could not find their Rocq source code. Their semantics is denotational and cannot easily cover all the features our paper [11] does, but would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by [2], and we thank Y. Bertot for insightful discussions on the topic. In [2], Bertot mechanizes the correctness of a compiler. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence. His approach uses the compiler as the translation function from one language to the other, but does not introduce a decompiler, just as we do not define a function from lists to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program. We adopt the same strategy in the proof of Theorem 13.

8 Conclusion

We do this. This is a first step for XXXX. We need to add substitutions. The missing part of the proof becomes related to abstract interpretation, since the det types for a lattice and one has to prove that the concrete runtime values variables take agree with the abstraction computed statically. This proof is ongoing and somehow orthogonal to the current one since the cut operator and the HO feature do not interfere with eachother in complex ways.

References

1 Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.

2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: https://inria.hal.science/inria-00073199v1.

3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.

4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.

5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34, doi:10.4230/LIPIcs.FSCD.2020.34.

6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:Debray1988.

7 Saumya K. Debray and David S. Warren. Functional computations in logic programs. volume 11, page 451–481. Association for Computing Machinery, July 1989. doi:10.1145/65979.65984.

8 Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: https://inria.hal.science/hal-01176856v1, doi:10.1007/978-3-662-48899-7_32.

9 Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: https://inria.hal.science/hal-04467855.

10 Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.

11 Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.

12 Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.

13 Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/S0960129518000427.

14 Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27`, `doi:10.4230/LIPIcs.ITP.2025.27`.

15 Jael Kriener and Andy King. Redalert: Determinacy inference for prolog. volume 11, pages 537–553. Cambridge University Press, 2011.

16 Lunjin Lu and Andy King. Determinacy inference for logic programs. volume 3444, pages 108–123, 04 2005. `doi:10.1007/978-3-540-31987-0_9`.

17 math-comp. finmap — finite maps for mathcomp, 2026. URL: `https://github.com/math-comp/finmap`.

18 Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.

19 Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, 2012.

20 Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

21 Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. volume 29, pages 17–64, 1996. High-Performance Implementations of Logic Programming Systems. URL: `https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684`, `doi:10.1016/S0743-1066(96)00068-4`.

22 SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-Prolog PlDoc reference. URL: `https://www.swi-prolog.org/pldoc/doc_for?object=!/0`.

23 Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λProlog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018. URL: `https://inria.hal.science/hal-01637063`.

24 Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPIcs*, pages 29:1–29:18, September 2019. URL: `https://inria.hal.science/hal-01897468`, `doi:10.4230/LIPIcs.CVIT.2016.23`.

25 Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi. Master's thesis, Radboud University Nijmegen, 2024. `doi:10.5281/zenodo.12568604`.

26 David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA, October 1983. URL: `https://www.sri.com/wp-content/uploads/2021/12/641.pdf`.

27 Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*, 12(1–2):189–218, January 2012. `doi:10.1017/S1471068411000445`.